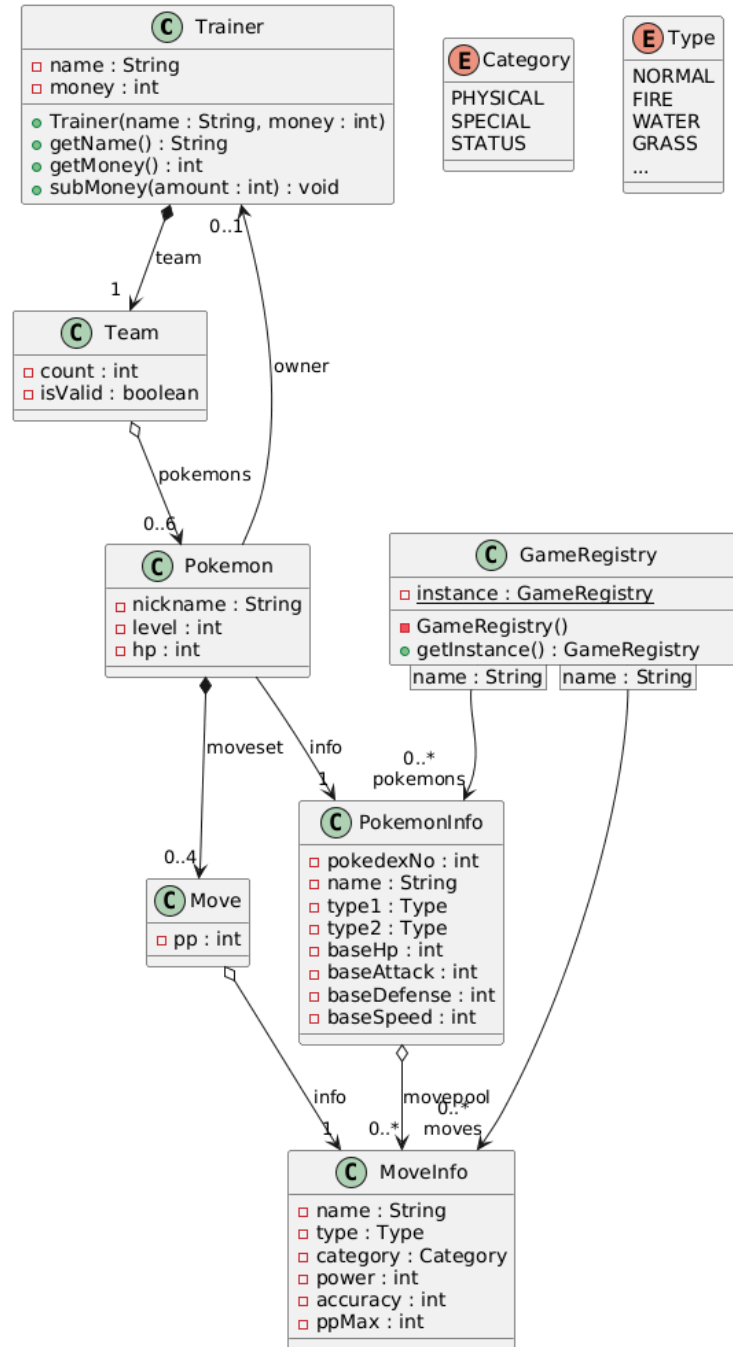


Class Diagram

David Darras

22/04/2026



NB: For the sake of clarity and brevity, constructors and standard methods (e.g., setLevel, getNickname, addMove, etc.) are omitted in the code below.

- **Encapsulation (Red Square/Green Circle):** All fields are **private** to protect the object's internal state. Interaction occurs via **public** methods, ensuring the principle of data hiding.
- **Navigability (Arrows):** Indicates the direction of the reference. A **Pokemon** "knows" its **Move** instances, but a **Move** has no reference to its owner (unidirectional).
- **Composition (Filled Diamond):** If a **Trainer** is destroyed, the **Team** is also destroyed.

- **Aggregation (Hollow Diamond):** If a Team is destroyed, the Pokemon instances survive and are sent back to the PC.
- **0..1 vs 1:** 0..1 indicates a nullable field (e.g., a wild Pokemon), while 1 indicates a mandatory reference where an exception should be thrown if missing.
- **Final Keyword:** Forces mandatory initialization in the constructor and prevents the reference from being changed later (immutability).
- **0..*** typically maps to a List<> or Map<>.
- **Enums:** Treated as primitive types for diagram legibility; no association lines are drawn.

```

1 public enum Category { PHYSICAL, SPECIAL, STATUS }
2
3 public enum Type { NORMAL, FIRE, WATER, GRASS, /* etc. */ }

```

```

1 public class GameRegistry {
2     private final Map<String, MoveInfo> moves;
3     private final Map<String, PokemonInfo> pokemons;
4
5     // Singleton Pattern
6     private static final GameRegistry instance = new GameRegistry();
7
8     private GameRegistry() {
9         this.moves = new HashMap<>();
10        this.pokemons = new HashMap<>();
11        // create all moves & pokemons
12    }
13
14    public static GameRegistry getInstance() { return instance; }
15 }

```

```

1 public class Trainer {
2     private final String name;
3     private int money;
4     private final Team team;
5
6     public Trainer(String name, int money) {
7         this.name = name;
8         this.money = money;
9         this.team = new Team();
10    }
11    public String getName() { return name; }
12    public int getMoney() { return money; }
13    public void subMoney(int amount) { this.money -= amount; }
14 }

```

```

1 public class Team {
2     private int count;
3     private boolean isValid;
4     private final Pokemon[] pokemons;
5
6     public Team() {
7         this.pokemons = new Pokemon[6];
8     }
9 }

```

```

1 public class Pokemon {
2     private String nickname;
3     private int level;
4     private int hp;
5     private final PokemonInfo info;
6     private Trainer owner; // can be null
7     private final Move[] moveset;
8
9     public Pokemon(PokemonInfo info) {
10        this.info = info;
11        this.moveset = new Move[4];
12    }
13 }

```

```

1 public class Move {
2     private int pp;
3     private final MoveInfo info;
4
5     public Move(MoveInfo info) {
6         this.info = info;
7     }
8 }

```

```

// Example : new MoveInfo("Tackle", Type.NORMAL, Category.PHYSICAL,
↪ 40, 100, 35);
1 public class MoveInfo {
2     private final String name;
3     private final Type type;
4     private final Category category;
5     private final int power;
6     private final int accuracy;
7     private final int ppMax;
8 }

```

```

// Example : new PokemonInfo(25, "Pikachu", Type.ELECTRIC,
↪ Type.ELECTRIC, 35, 55, 40, 90);
1 public class PokemonInfo {
2     private final int pokedexNo;
3     private final String name;
4     private final Type type1;
5     private final Type type2;
6     private final int baseHp;
7     private final int baseAttack;
8     private final int baseDefense;
9     private final int baseSpeed;
10    private final List<MoveInfo> movepool;
11 }
12

```